

```
test30=# SELECT * FROM ACCOUNT;
userid | firstname | lastname | password | emailid | created_on
-----+-----+-----+-----+-----+-----
      1 | john     | mathew  | john1    | john.mathew@coy.com | 2020-07-06 05:55:07.586781
      4 | alexi    | kalikov | alexi1    | alexi.kalikov@coy.com | 2020-07-06 05:55:09.529932
      2 | sam      | antony  | sam2     | sam.antony@coy.com | 2020-07-06 05:55:07.637664
      3 | imran    | arian   | imran3    | imran.arian@coy.com | 2020-07-06 05:55:07.647246
(4 rows)

test30=# SELECT * FROM ACCOUNT LIMIT 2;
userid | firstname | lastname | password | emailid | created_on
-----+-----+-----+-----+-----+-----
      1 | john     | mathew  | john1    | john.mathew@coy.com | 2020-07-06 05:55:07.586781
      4 | alexi    | kalikov | alexi1    | alexi.kalikov@coy.com | 2020-07-06 05:55:09.529932
(2 rows)

test30=# CREATE INDEX IDX2 ON ACCOUNT (USERID);
CREATE INDEX
test30=# SELECT * FROM ACCOUNT ORDER BY USERID ASC;
userid | firstname | lastname | password | emailid | created_on
-----+-----+-----+-----+-----+-----
      1 | john     | mathew  | john1    | john.mathew@coy.com | 2020-07-06 05:55:07.586781
      2 | sam      | antony  | sam2     | sam.antony@coy.com | 2020-07-06 05:55:07.637664
      3 | imran    | arian   | imran3    | imran.arian@coy.com | 2020-07-06 05:55:07.647246
      4 | alexi    | kalikov | alexi1    | alexi.kalikov@coy.com | 2020-07-06 05:55:09.529932
(4 rows)
test30=# \dt
          List of relations
Schema | Name      | Type  | Owner
-----+-----+-----+-----
public | account   | table | yugabyte
(1 row)
```

Figure 2: Standard queries in YSQL

After downloading the above YAML script, you can run the command given below in the AWS CLI (console interface) to create the YugaByteDB instance in the AWS Cloud subscription:

```
aws cloudformation create-stack
--stack-name <stack-name> --template-
body file://yugabyte_cloudformation.yaml
--parameters ParameterKey=DBVersion,Par
ameterValue=2.1.8.2,ParameterKey=KeyNam
e,ParameterValue=<ssh-key-name>
```

Layering the architecture of YugaByteDB

Since YugaByteDB is a NewSQL type of database providing RDBMS and NoSQL DB functionalities, it has two layers — one for each of these services — called the query layer and the Document Store (DocDB) layer, as shown in Figure 3.

Each layer is lightweight and flexible enough to adopt to API calls and service requests to handle transaction processing in the database.

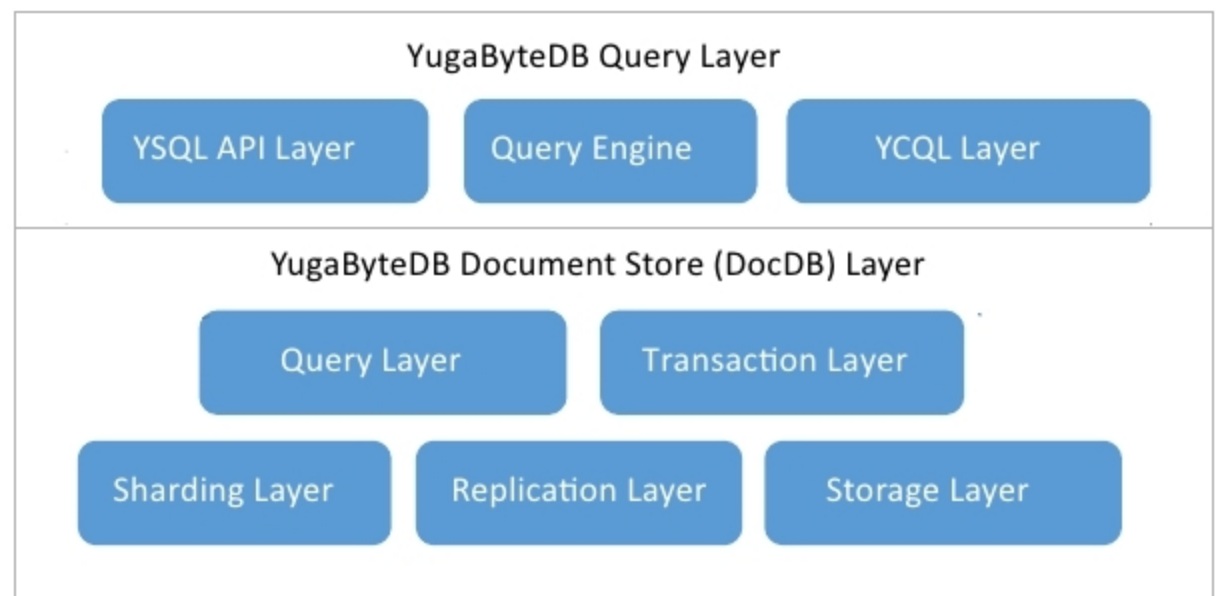


Figure 3: YugaByteDB core layers

DocDB interface

DocDB or the Document Store layer is the distributed document store for facilitating strong Database write consistency in transaction processing (a resilient feature for highly resistant DB operations), automatic sharding to improve query performance (partitions the DB on its own), and flexible tuning to improve data read consistency and speed.

Data in DocDB is stored in a key-value pair in the table, where the table consists of rows of data, each row comprising a key to identify the data and a document paired to it as a JSON or XML element (Figure 4).

YugaByteDB offers high performance with a rate of about 1 million read/writes per second consistently while processing large volumes of data, as

Continued to page 57...